AI-ASSISTED CODE AUTHORING AT SCALE

FINE-TUNING, DEPLOYING, AND MIXED METHODS EVALUATION

MURALI ET AL., PROC. ACM SOFTW. ENG. (FSE), 2024

Presented By: George Maison

Agenda

- Problem & motivation
- What is CodeCompose?
- Model + training objective (LCM)
- System architecture
- Results: accuracy, adoption, feedback
- Limitations & discussion questions

Motivation

- Large, fast-changing internal codebase → knowledge is hard to discover
- Developers spend time: boilerplate, API lookup, documentation
- Goal: real-time AI code suggestions inside IDEs

What is CodeCompose (Meta)

- AI-assisted code authoring tool deployed internally at Meta
- Supports 9+ languages (including internal ones like Hack/Flow)
- Inline suggestions; can use natural-language comments
- Designed for editing (uses code before AND after cursor)

Figure 1 — Inline suggestions, comment-aware completion, and bi-directional context


```
elif isinstance(model_dir, dict):
    self.model_dirs = [model_dir["bucket"] + "/" + model_dir["dir"]]
if len(set(self.model_dirs)) < len(self.model_dirs):
    raise ValueError("Duplicate model directories detected.")
```

(a)

```
elif isinstance(model_dir, dict):
    self.model_dirs = [model_dir["bucket"] + "/" + model_dir["dir"]]
if len(set(self.model_dirs)) < len(self.model_dirs):
    # Kill the service!
    sys.exit("Duplicate model paths found. Please specify only one model path")
```

(b)

```
def is_service_running(self) -> int:
    """
    Returns 0 if service is running else non-zero.
    """
    if self.running:
        return 0
    else:
        return self.error_code
```

(c)

Fig. 1. CodeCompose (a) offers inline code suggestions in VSCode in a grey text appearing after the cursor when the user is typing code (Tab to accept), (b) changes its suggestion to adapt to a natural language comment, (c) suggests code or documentation based on code below the current position.

its accuracy. The code before and some of the code after the masked part is also fed to the model as the context, along with some additional metadata. We contrast the public InCoder model [18], against our fine-tuned model with both the Causal Masking (CM) and Language Causal Masking (LCM) objectives.

Result Summary: We find that LCM CodeCompose model exactly recreates the masked line between 40% and 58% of the time and has a high BLEU score between 56% and 73%, depending on the programming language. This represents an improvement over the Public model between 1.4× and 4.1×.

RQ2. Adoption: How many suggestions are accepted by engineers and what proportion of the code is written by CodeCompose?

Few studies of LLM for coding are able to go beyond backtests. There are some blog posts demonstrating the use by professional engineers. At Meta we are able to gradually rollout new features to engineers. We progressively roll CodeCompose out and monitor its impact through the number of accepted suggestions and the proportion of code that is written by CodeCompose.

Result Summary: In this paper, we make over 4.5M suggestions to 16K engineers. We see an acceptance rate of 22% which is comparable to those at Google and GitHub. We see that 8% of the changed code is written by CodeCompose, which is higher than the 3% reported by Google. CodeCompose is rolled out to 100% of engineers at Meta.

RQ3. Developer Feedback: How do developers perceive CodeCompose in their daily work?

We triangulate our numerical analysis, which show evidence of adoption, with feedback from developers and discussion of actual usages [15]. Using the feedback group for CodeCompose, we

Source: Murali et al. (2024), Fig. 1 (paper page 4).

Research Questions (RQs)

- RQ1: Model evaluation — can it reproduce a hidden line of code?
- RQ2: Adoption — how often are suggestions accepted; how much code is AI-written?
- RQ3: Feedback — how do developers perceive the tool in daily work?

Training Objective: Language Causal Masking (LCM)

- Mask a code span at language level (not tokenizer level)
- Mask at trigger characters where completions happen: (, ., =, etc.
- Add metadata (language, file path, notebook kernel)
- Use 70/30 context split: before/after cursor

Figure 2 — How LCM constructs training inputs (metadata + before + after + target)

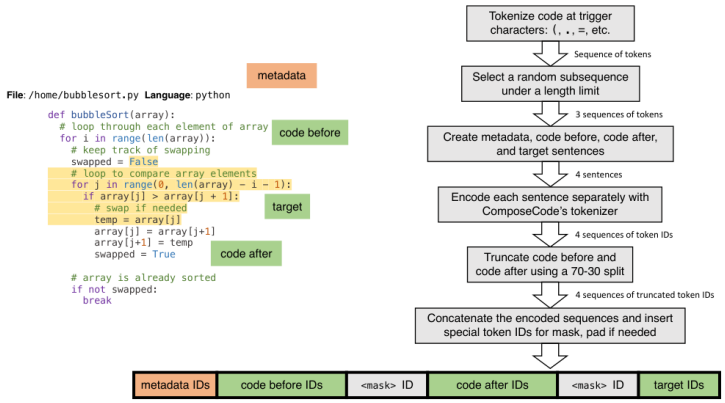

Fig. 2. Steps to construct an input to the model in LCM: (i) the code is tokenized at trigger characters where we expect the model to offer suggestions in production, (ii) a random subsequence of tokens is selected to be masked as the "target" to predict given the code before and the code after it, (iii) any additional metadata such as the filename is added to the front, (iv) all four strings (metadata, code before, target, code after) are encoded into tokens, (v) since the model's input length is limited, a 70-30 split is applied to the code before and code after if needed, (vi) all tokens are concatenated together into a single list of tokens with special tokens added to denote the masked target portion. The code on the left shows an example with the randomly selected target portion highlighted.

conduct a thematic analysis of the comments from 70 engineers. We provide direct quotes from engineers to contextualize and ground the themes we discovered. We also use these themes and feedback to iteratively improve CodeCompose.

Result Summary: The feedback was overwhelmingly positive, with only 8.5% of comments containing negative feedback and less than 1% of engineers turning off CodeCompose. The top two feedback comment themes related to discovering APIs and dealing with boilerplate (23%), and accelerating coding (20%). Meta continues to invest in CodeCompose efforts to iteratively improve the AI-assisted authoring experience.

The paper is organized as follows. Section 2 presents details about the CodeCompose model. Section 3 presents the system architecture. Section 4 presents results from our large-scale deployment at Meta. Section 5 discusses threats to validity. Section 6 discusses related work. Section 7 discusses the challenges for coding assistants at scale. Section 8 concludes the paper.

2 MODEL DEVELOPMENT AND EVALUATION METHODOLOGY

In this section, we provide details about the underlying LLM architecture for CodeCompose, and discuss considerations that affected our decision.

2.1 Model Architecture and Training Objective

LLMs largely fall into two main categories. The BERT [16] branch of models are trained with the Masked Language Model (MLM) objective that masks out certain parts of the input and trains the

Source: Murali et al. (2024), Fig. 2 (paper page 5).

Training Data & Fine-Tuning

- Base model: InCoder-1.3B (public code + StackOverflow)
- Fine-tuned on Meta first-party code (diffs, last ~2 years, production code)
- Trained across 9 languages; deployed on GPU inference cluster

System Design (High level)

- Client-server architecture for low-latency inline completions
- IDE client → Language Server Protocol (LSP) → GPU inference service
- Telemetry logs usage and latency for monitoring and evaluation

Figure 3 — CodeCompose system architecture (clients, LSP, inference, telemetry)

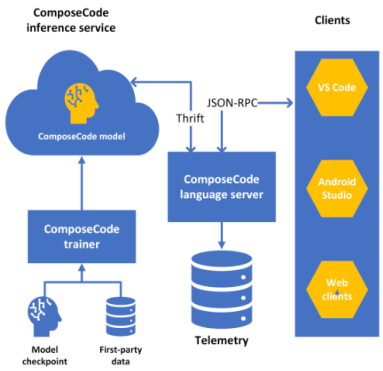

Fig. 3. System Architecture for CodeCompose. A pre-trained model checkpoint (InCoder) is trained by the trainer on first-party company data to create the fine-tuned CodeCompose model. This is served in a cluster of inference service machines with GPUs. When clients (VS Code, IDEs, web surfaces) require a code suggestion they send a JSON-RPC request to the CodeCompose language server protocol (LSP), which in turn communicates with the inference service through language-agnostic Thrift calls and sends the generated suggestion back. The LSP also logs telemetry which allows us to compute usage metrics, run experiments, and monitor for regressions. Clients, e.g., VS Code, then display the suggestion through their respective editors.

To evaluate the model, we run different versions of the model including the InCoder model as the public model and the fine-tuned model on the first-party data with the CM objective as well as the fine-tuned one with LCM objective. After running the different versions of the CodeCompose model, we measured two metrics and broke them down into the four main languages of Python, C++, Hack, and JavaScript to evaluate how well the model performs and the impact of fine-tuning with first-party data from Meta on the model accuracy. We use the measures of Exact Match (EM), as our most restricted metric, and BLEU score [30], which are used in similar studies [18, 27] and measured by comparing the reference of the masked part of the code and the predicted tokens by the model. The detailed results of this evaluation is described in Section 4.1.

3 SYSTEM DESIGN

The components of the CodeCompose system are shows in Figure 3. CodeCompose employs a typical client-server architecture in which the *server* is an inference tier that runs the model and the *client* is an editor that surfaces code suggestions, including VS Code and Android Studio. We encode the bulk of the client-side logic in a Language Server Protocol (LSP) [8] conformant language server that is reused across multiple editor integrations.

3.1 Server

At Meta, we have a tier of machines equipped with A100 GPUs, each with sufficient memory to host the fine-tuned CodeCompose model. Clients can make requests to this tier via Thrift [34]. The caller specifies the code before and after the cursor, as well as the file path, language, and metadata to use to process the request. The request handler is designed to be simple to utilize the most out of

Source: Murali et al. (2024), Fig. 3 (paper page 8).

Table 1 — Offline model accuracy (Exact Match & BLEU) across languages

Table 1. Outcome metrics across programming languages for the Public model and the fine-tuned models. Our LCM fine-tuned model substantially outperforms the Public model by between 1.4 times and 4.1 times.

Percentage of Exact Matches (improvement over Public Model by × times)								
Models	Python		Hack		Flow		C++	
Public	23.1%	–	14.2%	–	18.6%	–	23.4%	–
Fine-tuned (CM)	35.5%	1.5×	37.8%	2.7×	30.8%	1.7×	24.2%	1.0×
Fine-tuned (LCM)	48.8%	2.1×	57.7%	4.1×	52.2%	2.8×	40.0%	1.7×
BLEU Score (improvement over Public Model by × times)								
Models	Python		Hack		Flow		C++	
Public	37.9%	–	27.6%	–	33.8%	–	39.9%	–
Fine-tuned (CM)	50.0%	1.3×	52.6%	1.9×	45.8%	1.4×	40.3%	1.0×
Fine-tuned (LCM)	62.0%	1.6×	73.0%	2.6×	68.6%	2.0×	56.2%	1.4×

This experiment provides strong evidence that an internally fine tuned model outperforms an off-the-shelf model that is trained on external data only. The LCM model is very successful and is used in the subsequent sections where we roll out CodeCompose to developers.

> We find that LCM CodeCompose model exactly recreates the masked line between 40% and 58% of the time and has a high BLEU score between 56% and 73%, depending on the programming language. This represents an improvement over the Public model between 1.4× and 4.1×.

4.2 RQ2. Adoption Results

How many suggestions are accepted by engineers and what proportion of the code is written by CodeCompose?

We discuss the design of our CodeCompose system and its integration with VS Code and other editors used at Meta in Section 3. All changes at Meta must be gradually rolled out to users and we discussed our rollout strategy and adoption metrics in Section 3.5. Table 2 shows the adoption rates of CodeCompose.

Table 2. The acceptance rate and the proportion of code that was written by CodeCompose across programming languages

Language	Code shown	Acceptance rate	Code by CodeCompose	# Users
Python	1.9M	22%	8%	10.7K
Hack	1.3M	22%	10%	5.5K
C++	608.1K	20%	10%	2.5K
Flow (Javascript) [7]	583.2K	18%	7%	2.5K
Rust	74.2K	17%	9%	212
Objective C++	57K	18%	6%	429
Objective C	34K	18%	6%	299
C	23.5K	21%	12%	201
Typescript	8.9K	19%	10%	76
Total	4.5M	22%	8%	16K

Source: Murali et al. (2024), Table 1 (paper page 11).

Table 2 — Adoption by language (acceptance rate & code share)

CodeCompose made 4.5 million suggestions across 9 programming languages. 16 thousand distinct developers have seen at least one CodeCompose suggestion. As shown in Table 2, we observe a suggestion level acceptance rate of 22% across 9 programming languages for suggestions that were displayed for at least 750 milliseconds. Imposing a lower bound on the display time of the suggestions when calculating the acceptance rate helps in making sure developers who were exposed to a suggestion had a chance to see and comprehend it [5].

Our rollout strategy started with Python, which explains why there are more recommendations in that language. However, even typescript has over 8.9K recommendations and an acceptance rate of 19%, which is a substantial sample from a statistical perspective. CodeCompose has been rolled out to 100% of engineers at Meta.

We also log the length of each accepted suggestion, and the number of characters developers type when authoring code (excluding large copy-pastes, refactors, etc.). This allows us to calculate, at a character level, the percentage of the code typed by developers that came through accepting CodeCompose suggestions, which we computed to be 8%. CodeCompose is writing a substantial portion of the codebase, and in the next section we use developer feedback to understand what types of code are being accepted.

> We make over 4.5M suggestions to 16K engineers. We see an acceptance rate of 22% which is comparable to those at Google and GitHub. We see that 8% of the changed code is written by CodeCompose, which is higher than the 3% reported by Google. CodeCompose is rolled out to 100% of engineers at Meta.

4.3 RQ3. Developer Feedback Results

How do developers perceive CodeCompose in their daily work?

At Meta each internal tool has a feedback forum. The first two authors conducted a thematic analysis of the responses from 70 engineers (the methodology is described in Section 3.5). Similarly, the model does not differentiate between suggestion types such as code suggestions and documentation suggestion when making suggestions. We report the frequency of themes and suggestion types in Table 3. In subsequent sections, we provide grounded evidence of the themes through quotations and discuss both positive and negative feedback.

Table 3 lists the distribution of qualitative feedback responses. 91.5% of the CodeCompose users gave a favorable response while 8.5% of them gave an unfavorable response. Many respondents (15.7%) appreciated the fact that CodeCompose suggestions are accurate and CodeCompose was able to auto-complete the code they were going to type.

In addition to the accuracy of suggestions, 23% of the users said CodeCompose helped them discover new APIs, automating tedious tasks such as boilerplate code. Therefore, CodeCompose is not just helping with typing code faster (as demonstrated in Table 2) but also saving time and effort people spend searching for APIs and documentation.

CodeCompose is helping people move faster with with 20% of the users stating that they found CodeCompose to accelerating their coding activity. In addition to helping write code faster, CodeCompose is helping developers produce more in-code documentation and API documentation. This is an interesting side-effect and demonstrates the model's ability to generate text as an output from code in addition to code generation.

4.3.1 Evidence of Positive Experiences. After analyzing all the qualitative feedback, we identified the following main factors that make a developer inclined towards finding a system like CodeCompose useful.

Source: Murali et al. (2024), Table 2 (paper page 11–12).

Table 3 — Themes from developer feedback (n=70)

Table 3. Frequency of themes coded from the feedback of 70 engineers. There are only six unfavorable responses, so we discuss each in Section 4.3.2

Theme	Description	Engineers
Accelerate coding	CodeCompose helped speed up my coding process	14 (20.0%)
Discovery	CodeCompose helped me discover APIs, write boilerplate code faster	16 (22.8%)
Documentation	CodeCompose helped me with generating in-code documentation, docstrings, etc.	5 (7.1%)
Suggestion accuracy	CodeCompose suggestions are accurate and not noisy	11 (15.7%)
Generally favorable	CodeCompose is generally great, and I will keep using it for my coding tasks	18 (25.7%)
Unfavorable	CodeCompose is not useful	6 (8.5%)
Total		70 (100%)

Favorable scenarios for CodeCompose: The scenarios for which CodeCompose was able to add the biggest value includes (but is not limited to), auto-completing lines of code, API discovery, boilerplate coding, suggesting standard libraries, generating in-code documentation, etc.

When it comes to developers, the developers who benefited the most are the ones who work on authoring code that involves writing boilerplate code and idioms, the ones who follow the typical coding patterns employed at Meta, and the ones that employ standard first-party (or third-party) libraries to accomplish their coding tasks. Additionally, CodeCompose is well received by the developers who tend to work on building pipelines and common infrastructure tasks that involve writing heavily templatized code. This can be corroborated by the distribution of responses listed in Table 3 and the representative quotes listed below.

Grounded evidence of positive experiences. We share quotations that exemplify the positive experiences of developers. A developer who got access to CodeCompose shared their first impressions:

> *"Today I needed to write a quick tool for* [task]. *I opened VS Code and started writing* [code]. *Suddenly this thing called CodeCompose popped up and wrote all the code for me! This was a super delightful experience, it really made my day, so thank you!"*

In this case, the developer was unaware of the existence of CodeCompose. This quote summarizes the delightful coding experience that the developers at Meta experienced when they pair-program with CodeCompose.

Several remarks are clear indicators of the usefulness of employing CodeCompose in code authoring workflows.

> *"I have been a big fan of* [CodeCompose] *ever since I got access. I do not think I have been in any coding session since then where I did not use it."*

> *"The suggestions* [from CodeCompose] *are quite relevant and do not show up too often – not necessarily "rarely", but not obnoxiously often either. It is a nice balance."*

Source: Murali et al. (2024), Table 3 (paper page 13).

Key Results (Summary)

- RQ1: LCM exact-match recreates hidden line ~40–58% (varies by language)
- RQ2: 4.5M suggestions to 16K developers; ~22% acceptance; ~8% of code from AI
- RQ3: 91.5% feedback positive; top themes: API discovery/boilerplate and faster coding

Limitations / Threats to Validity

- Generalizability: results are from Meta's internal environment
- Metrics: exact match and BLEU may miss "useful but edited" suggestions
- Hallucinations + signature mismatches can reduce trust
- UX conflicts with traditional autocomplete (competition for Tab)

Conclusion

In conclusion, this paper demonstrates that AI-assisted coding tools can significantly improve developer productivity.

By combining large language models with real-world deployment and developer feedback, the authors show how AI can be integrated into the software development workflow.

The study also highlights important challenges such as trust, accuracy, and user experience.

QUESTIONS?

What Has Actually Changed Since the Paper

Aspect	In the Paper	Today
Model	InCoder (1.3B parameters)	GPT-4, Claude, Llama 3 etc
Tool	CodeCompose	Metamate / Devmate
Suggestions	mostly single-line	multi-line + agents
Capability	autocomplete	debugging, refactoring, code agents